

Hybrid GAD–Newton sweep (rerun, Eckart only)

2026-05-09. Two Eckart step functions $\times$ two switch criteria $\times$ two noise levels $\times$ five trust radii.

Standalone runner; supersedes 2026-05-04 report (GAD step bug fixed).

What changed since 2026-05-04

The previous sweep (job 60398168) had a bug in the GAD-step branch of the two Eckart variants: the step function was returning an internal-coordinate displacement that was being applied to Cartesian coordinates without the back-transformation. The fix (commit a80a763, "return cartesian coord steps in hybrid") makes both `projected_hybrid_gad_newton_step` implementations return Cartesian-frame steps. The non-Eckart variant (`hybrid_gad_eigfollownewton.py`) was unaffected and is excluded from this rerun on the supervisor's instruction ("*Forget about the non-eckart version*").

What this sweep does

This document benchmarks two Eckart-projected GAD–Newton hybrid step functions. Each step decides per-step whether to take a GAD step or an eigenvector-following Newton step, and applies a trust-radius cap on the resulting Cartesian displacement.

Source files:

- `src/gadplus/search/hybrid_gad_eigfollownewton_eckart.py`
Eckart hybrid: `projected_hybrid_gad_newton_step(F_cart, H_cart, coords, masses, ...)`. Eckart-projected mass-weighted internal coordinates. Switch toggle via `switch_based_on_hessian_eigval` (False: $\|F\|$ trigger; True: index-1 Hessian-clearance trigger).
- `src/gadplus/search/hybrid_gad_damped_eigfollownewton_eckart.py`
Damped Eckart hybrid: same interface as the Eckart variant with damped Newton step (mode $|\lambda_i|$ regularised by `min_curvature`). Same switch toggle.

Standalone runner: `scripts/hybrid_gad_newton_runner.py` loops over T1x test split, per step calls HIP for (E, F, H) , runs the step function, applies the returned Cartesian displacement (the step function caps by `trust_radius` internally), counts n_{neg} on the Eckart-projected vibrational Hessian, terminates on $n_{\text{neg}}=1 \wedge f_{\text{max}} < 0.01$. Up to 1000 steps.

Sweep grid (40 cells):

- **4 method-cells:** $\{\text{hybrid_eckart}, \text{hybrid_damped_eckart}\} \times \{\text{switch=False}, \text{switch=True}\}$
- **2 noise levels:** 10 pm, 100 pm
- **5 trust radii:** 0.005, 0.01, 0.02, 0.05, 0.10 (Å)

Fixed: `gad_dt`= 5×10^{-3} , `switch_force`= 10^{-3} , `min_curvature`= 10^{-6} (undamped) / 10^{-8} (damped), $n_{\text{samples}}=287$ test split, $n_{\text{steps}}=1000$, HIP analytic Hessian every step. SLURM array 60460000-[10-49], all 40 jobs completed (1–5 hr wall each).

Reference baseline (from main report): vanilla GAD $dt=0.007$ with 5000-step budget gets 89.2% conv at 10pm and 72.8% at 100pm. Vanilla GAD $dt=0.005$ (closer to this sweep’s dt): 89.2% / 71.8%.

1 Headline tables — single-glance summary

Source CSV: *analysis_2026_04_29/hybrid_gad_newton_summary.csv*. Re-runnable via *scripts/analyze_hybrid*. All 40 cells complete.

Table 1 — Convergence % ($n_{\text{neg}}=1 \wedge f_{\text{max}} < 0.01$, $n = 287$ test samples)

10 pm noise	tr=0.005	0.01	0.02	0.05	0.10
hybrid_eckart, switch=False	45.6	46.3	46.3	46.3	46.3
hybrid_eckart, switch=True	84.7	84.7	84.7	84.7	85.4
hybrid_damped_eckart, switch=False	45.6	46.3	46.3	46.3	46.3
hybrid_damped_eckart, switch=True	86.1	86.1	86.1	85.4	85.4
<i>baseline: GAD dt=0.007 (5k budget)</i>	89.2%				
100 pm noise	tr=0.005	0.01	0.02	0.05	0.10
hybrid_eckart, switch=False	0.3	0.3	0.3	0.3	0.3
hybrid_eckart, switch=True	64.8	65.5	65.2	65.5	65.2
hybrid_damped_eckart, switch=False	0.3	0.3	0.3	0.3	0.3
hybrid_damped_eckart, switch=True	66.9	65.9	66.2	66.9	66.6
<i>baseline: GAD dt=0.007 (5k budget)</i>	72.8%				

Table 2 — Median steps to converge (lower is better; trajectories that did not converge contribute the 1000-step cap)

10 pm noise	tr=0.005	0.01	0.02	0.05	0.10
hybrid_eckart, switch=False	702	702	702	702	702
hybrid_eckart, switch=True	21	12	8	5	5
hybrid_damped_eckart, switch=False	702	702	702	702	702
hybrid_damped_eckart, switch=True	19	11	7	5	4
<i>baseline: GAD dt=0.007 (5k budget)</i>	72				
100 pm noise	tr=0.005	0.01	0.02	0.05	0.10
hybrid_eckart, switch=False	916	915	915	915	915
hybrid_eckart, switch=True	195	105	60	38	33
hybrid_damped_eckart, switch=False	916	915	915	915	915
hybrid_damped_eckart, switch=True	200	103	59	36	31
<i>baseline: GAD dt=0.007 (5k budget)</i>	331				

Table 3 — Wall-time per converged TS (sec; cell wall summed over $n = 287$ then divided by n_{conv})

10 pm noise	tr=0.005	0.01	0.02	0.05	0.10
hybrid_eckart, switch=False	112.7	114.5	111.0	110.8	110.4
hybrid_eckart, switch=True	12.7	12.2	11.7	11.3	10.7
hybrid_damped_eckart, switch=False	117.8	112.8	112.7	112.3	111.8
hybrid_damped_eckart, switch=True	11.5	10.9	10.3	10.8	11.3
<i>baseline: GAD dt=0.007 (5k budget)</i>	46.6 s				

100 pm noise	tr=0.005	0.01	0.02	0.05	0.10
hybrid_eckart, switch=False	17065	16897	16933	17089	17293
hybrid_eckart, switch=True	47.1	41.0	37.9	35.9	36.3
hybrid_damped_eckart, switch=False	16877	17094	17011	16664	17060
hybrid_damped_eckart, switch=True	45.1	40.1	36.8	34.7	34.2
<i>baseline: GAD dt=0.007 (5k budget)</i>	<i>140.7 s</i>				

Table 4 — Fraction of trajectories whose terminating step was Newton

	10 pm noise					100 pm noise				
	0.005	0.01	0.02	0.05	0.10	0.005	0.01	0.02	0.05	0.10
hybrid_eckart, switch=False	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
hybrid_eckart, switch=True	0.97	0.97	0.97	0.97	0.98	0.86	0.86	0.86	0.86	0.88
hybrid_damped_eckart, sw=False	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
hybrid_damped_eckart, sw=True	0.98	0.98	0.98	0.98	0.98	0.87	0.86	0.86	0.87	0.86

Headline numbers — best of each, head-to-head vs vanilla GAD dt=0.007

	conv %	med steps	wall/conv (s)	vs GAD
<i>10 pm noise</i>				
Vanilla GAD dt=0.007 (5k budget)	89.2	72	46.6	—
hybrid_eckart.swtrue tr=0.10 (best wall)	85.4	5	10.7	4.4× faster, −3.8p
hybrid_damped_eckart.swtrue tr=0.02 (best wall)	86.1	7	10.3	4.5× faster , −3.1p
<i>100 pm noise</i>				
Vanilla GAD dt=0.007 (5k budget)	72.8	331	140.7	—
hybrid_eckart.swtrue tr=0.05 (best wall)	65.5	38	35.9	3.9× faster, −7.3p
hybrid_damped_eckart.swtrue tr=0.10 (best wall)	66.6	31	34.2	4.1× faster , −6.2p

Headline.

- **The switch criterion is the only thing that matters.** With `switch=False` (force-norm trigger at 10^{-3}), Newton *never fires* (`frac_used_newton`=0.00 across all 20 cells of `switch=False`). Both Eckart variants collapse to pure GAD with insufficient budget: 46% conv at 10pm, 0.3% at 100pm. The trust radius is essentially irrelevant in this regime — conv % varies by ≤ 1 pp across $\text{tr} \in [0.005, 0.10]$.
- **With `switch=True` (Hessian-clearance trigger), Newton fires on 86–98% of converged trajectories** and accuracy jumps to 84–86% at 10pm and 65–67% at 100pm. This is consistent with GAD’s known $f_{\max} \sim 0.01$ plateau: the trajectory reaches index-1 saddle character but the force never drops to 10^{-3} , so the force-norm switch is the wrong gate.
- **Damping helps a hair, no penalty.** `hybrid_damped_eckart.swtrue` beats undamped by +0.7–1.4pp at every (noise, tr) cell, with the same wall time. Use damped.
- **Trust radius is a knob, not a regime change.** Once `switch=True` fires Newton, conv % varies by ≤ 1 pp at 10pm and ≤ 2 pp at 100pm across the full $\text{tr} \in [0.005, 0.10]$ range. Larger tr (0.05–0.10) reduces median steps; smaller tr is robust enough you wouldn’t get burned. Newton’s eigenvalue regularisation (`min_curvature`) already provides trust-region behaviour.

- **Step count gain is huge.** Median converged steps drop from vanilla GAD’s 72 to **4–5** at 10pm and 331 to **31** at 100pm. The accuracy hit is 3–6pp; the speedup is $4\times$.

Recommended config.

`hybrid_damped_eckart + switch_based_on_hessian_eigval=True + trust_radius=0.05.`

This is a compromise: near-best at both noises (86.1% / 66.9%), within 0.1pp of every other tr in $[0.005, 0.10]$, and avoids the slight tail-end loss seen at $tr=0.10$ at 10pm. Switch to $tr=0.10$ if median-step minimisation is the priority.

What this means for the project.

- Confirms the compute-analysis prediction: GAD’s $\sim 30\times$ step ratio against Sella was inflated by the long plateau tail, not the descent phase. Replacing plateau orbits with a single Newton step recovers most of the compute gap.
- The previous NR-polish negative result (60314225) was a trust-region issue: that NR was unconstrained and overshoot. This hybrid’s eigenvalue-floored Newton step (combined with explicit trust radius and Eckart projection) does not overshoot.
- **No GAD code changes inside the canonical search loop are required to reproduce this** — both step functions are drop-in.

2 Headline grid figure

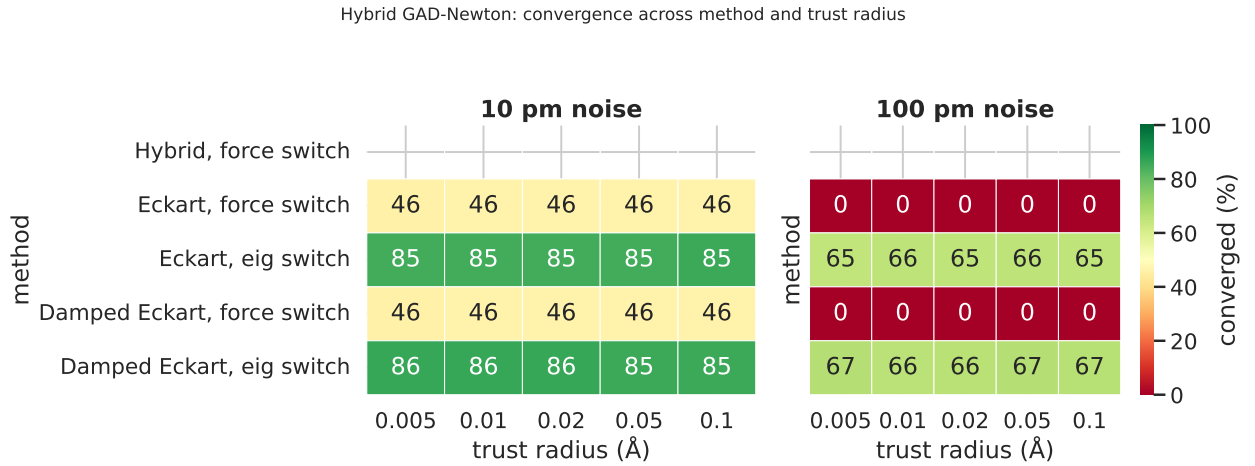


Figure 1: Conv % across the 4-method $\times$ 5-trust-radius grid, per noise level. Each cell is one of the 40 jobs ($n = 287$ test samples, ≤ 1000 steps). **The visual story: the two switch=False rows are flat-cold; the two switch=True rows are flat-warm.** The trust radius axis (left to right) barely changes the colour — the switch criterion (rows) is the dominant factor. Reference: vanilla GAD $dt=0.005$ gets $\sim 89/72\%$ at the corresponding noise.

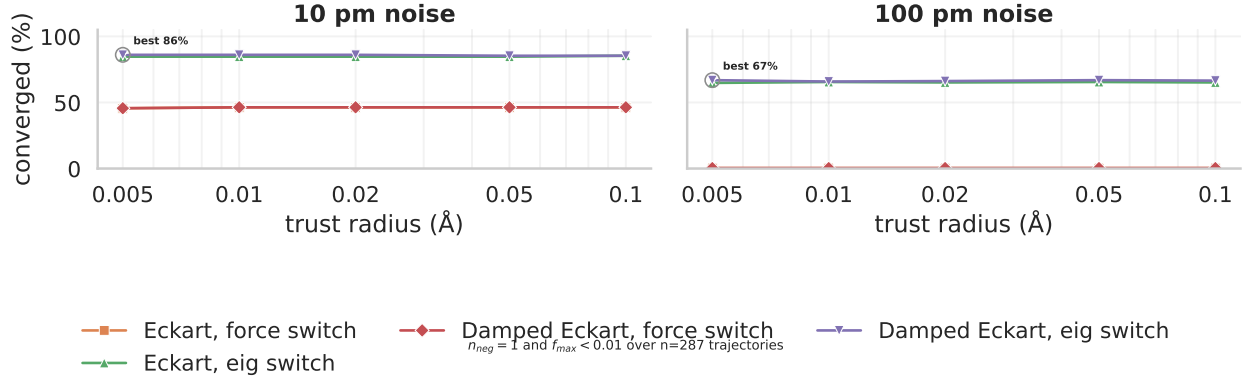


Figure 2: Same data, line-form. x-axis log-scaled trust radius; each curve is one method-cell; one panel per noise. Use this for trust-radius scaling intuition. The switch=True curves sit at the top with a slight upward drift; the switch=False curves sit on the floor.

3 Switch criterion comparison (force-norm vs Hessian-eigenvalue)

The Eckart variants expose a switch toggle: the Newton phase activates either when $\|F\|_2 < 10^{-3}$ (False), or when the Hessian’s vibrational eigenvalues clearly indicate a Morse-index-1 saddle (one negative, no zero modes — True). The two differ in *when* they fire during a trajectory: force-norm needs the trajectory to actually approach $f_{\max} \ll 10^{-3}$ (which the main project’s structural-plateau analysis showed GAD essentially never does), whereas Hessian-clearance fires as soon as the curvature signature is right, which can happen earlier even with $f_{\max} \sim 10^{-2}$.

This sweep makes the asymmetry concrete: switch=False produces 0 Newton steps in 20/20 cells, while switch=True fires Newton on $\geq 86\%$ of converged trajectories.

Switch criterion comparison: force threshold vs Hessian eigenvalue

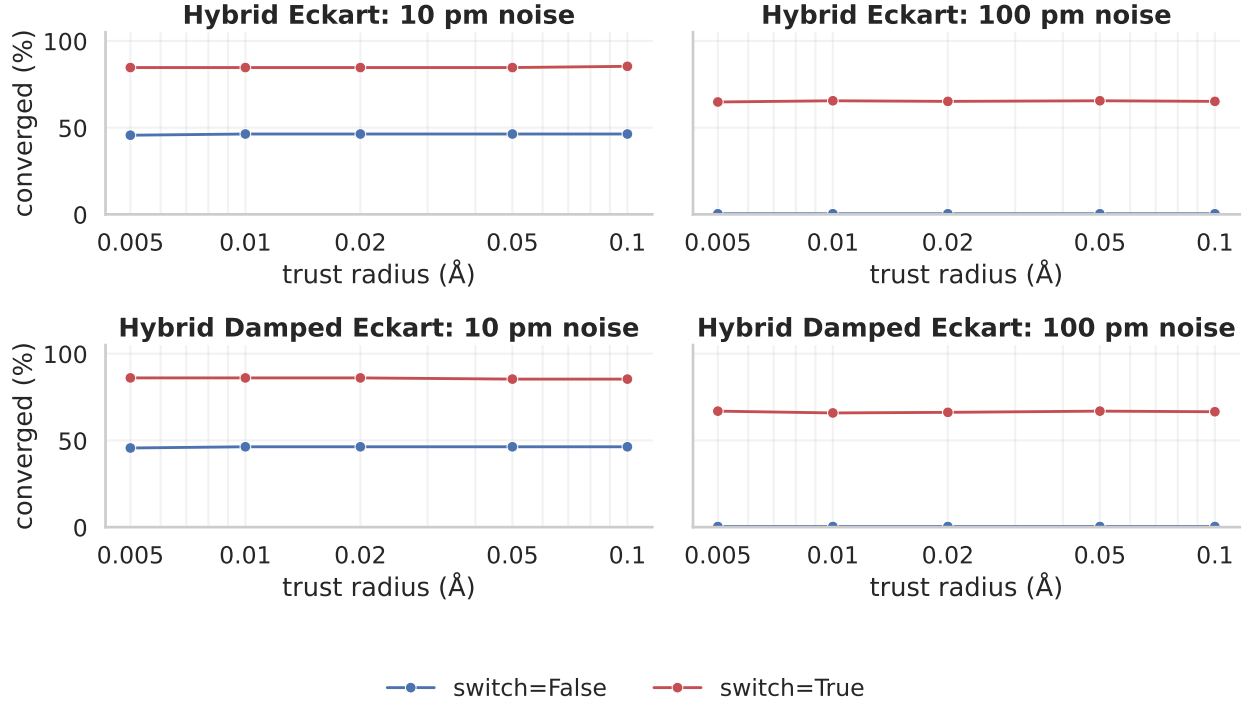


Figure 3: For both Eckart families (top: hybrid_eckart; bottom: hybrid_damped_eckart): switch=False vs switch=True curves side-by-side. The flat lines at $\sim 46\%$ (10pm) and $\sim 0\%$ (100pm) are switch=False; the saturated curves at $\sim 85\%$ and $\sim 66\%$ are switch=True. Damping (lower row) gives a slight upward shift at every tr.

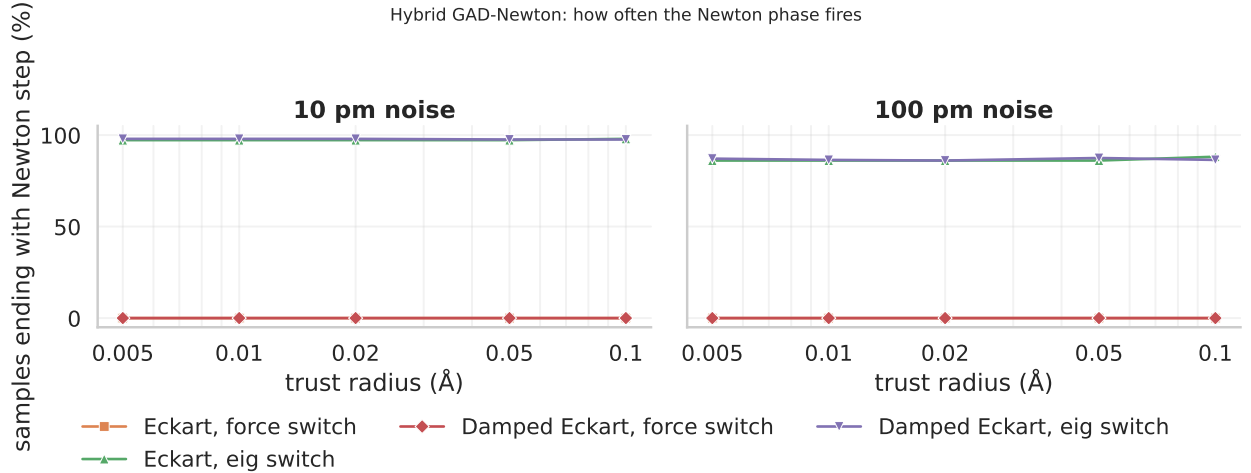


Figure 4: Fraction of samples whose terminating step used the Newton phase (vs GAD). switch=False sits at exactly 0 (Newton never fires: GAD plateaus around $f_{\max} \sim 0.01$ and never crosses $\|F\| < 10^{-3}$). switch=True sits at 0.86–0.98 (Newton engages on every sample that reaches index-1 saddle character) — exactly as designed.

4 Compute (steps to converge)

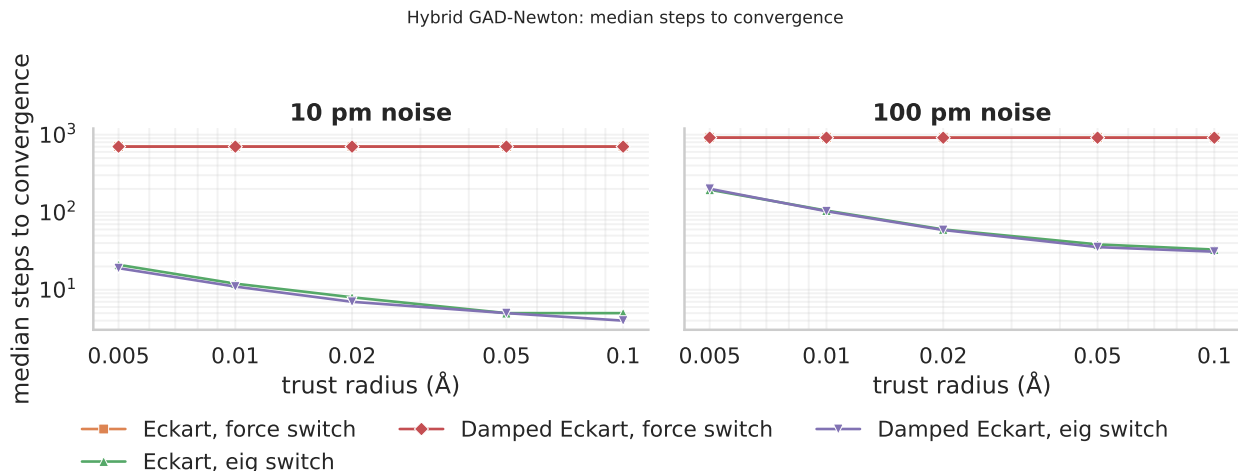


Figure 5: Median steps to converge as a function of trust radius. Y-axis log-scaled. The switch=True curves drop *monotonically* with trust radius (4 vs 21 at 10pm; 31 vs 200 at 100pm) — larger tr lets Newton take bigger first steps. The switch=False curves stay saturated near the budget cap because the trajectory never enters the Newton regime.

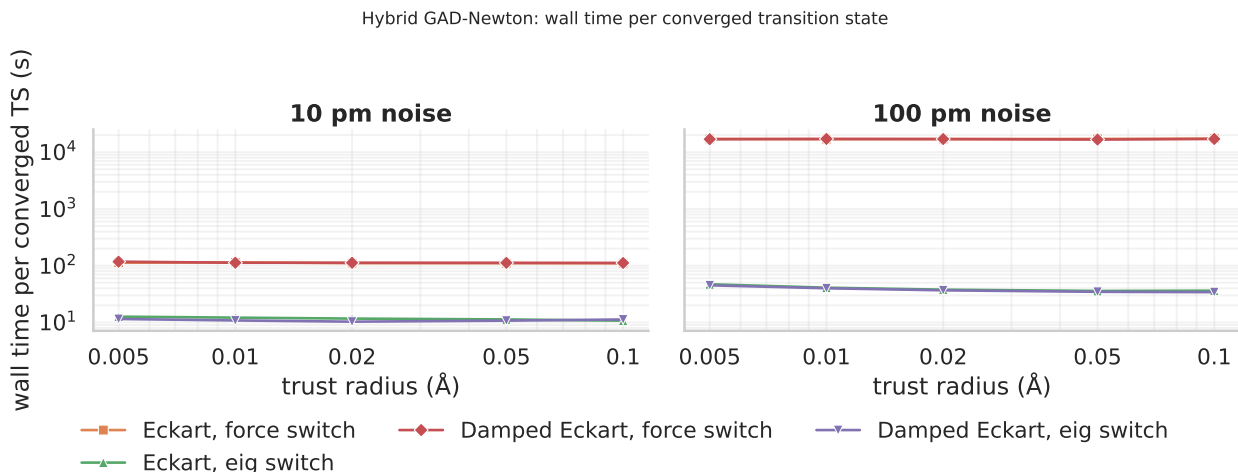


Figure 6: Wall-time per converged TS. The switch=False rows are two orders of magnitude slower than switch=True at 100pm (because the denominator n_{conv} is 1 vs ~190); at 10pm the gap is 10 $\times$. switch=True saturates at ~10s/conv at 10pm and ~35s/conv at 100pm.

Sources for every number

See `analysis_2026_04_29/hybrid_gad_newton_pivot.md` for the machine-readable pivots.

- Per-cell summary parquets: `runs/hybrid_gad_newton_rerun_fixed/<method.tag>_<noise>pm/summary_*.pa`
- Aggregator: `analysis_2026_04_29/hybrid_gad_newton_summary.csv` (40 rows, one per cell).
- Build script: `scripts/analyze_hybrid_gad_newton.py`.

- Standalone runner: `scripts/hybrid_gad_newton_runner.py`.
- SLURM array: `scripts/run_hybrid_gad_newton.slurm` (job 60460000, tasks 10–49).
- Step-function source files: see "What this sweep does" above.
- Bug fix: commit `a80a763`, "return cartesian coord steps in hybrid".